

Identity Command Center (ICC)

A complete product design case study covering end-to-end UX redesign, design system architecture, and developer handoff — for BeyondID's enterprise identity platform built on Okta and Auth0.

COMPANY

BeyondID

MY ROLE

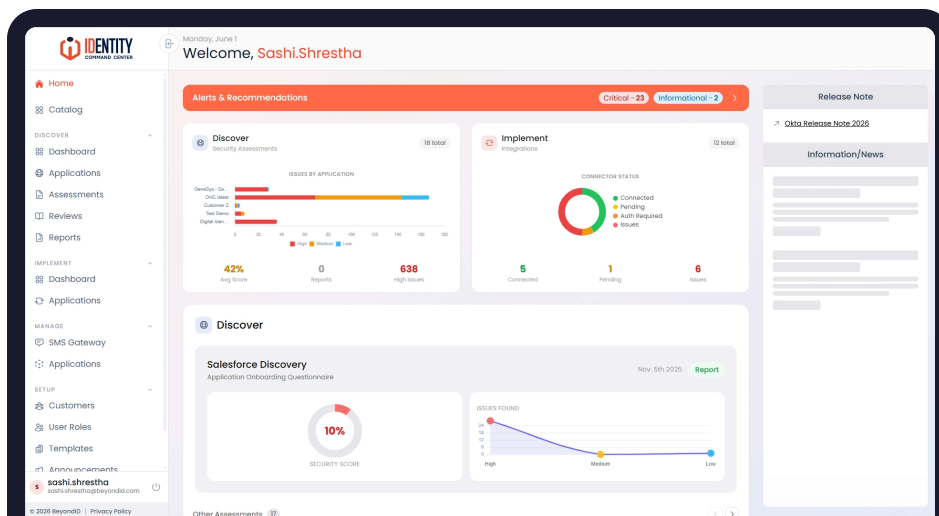
Senior UI/UX
Designer & Front-
End Developer

DOMAIN

Identity & Access
Management

STACK

Figma · NextJS
React · Tailwind ·
Headless UI ·
Heroicon



3+

Major screens
redesigned end-to-end

16

UX issues identified &
resolved

8

Dev-ready component
files shipped

100%

Design token coverage

CONTENTS

- 01 Product overview & context
- 02 Design system foundation
- 03 Health Check Assessment
- 04 Manage Report Design
- 05 Developer handoff system
- 06 Impact & key learnings

01 Product overview & context

What is the ICC?

The Identity Command Center (ICC) is BeyondID's flagship enterprise SaaS platform — a centralized hub for managing identity configurations, running compliance assessments, and monitoring tenant health across Okta and Auth0 environments. It's built for security engineers, IAM architects, and compliance officers who work in high-stakes environments where interface clarity directly affects the security posture of their client organizations.

The opportunity I was brought in to solve

ICC had grown screen by screen — each page was functional but built in isolation. There was no unified visual language, no shared component library, and no token system connecting design to code. The product worked, but it didn't feel like a cohesive product. My mandate was to establish the design foundation and then systematically elevate each critical screen on top of it.

My position was unusual and powerful: *I was both the designer and the front-end developer on the product. I didn't hand off designs for someone else to interpret — I built the components I designed. Every decision was grounded in what was actually shippable.*

My responsibilities



End-to-end UX redesign of the Health Check Assessment and Manage Report screens — from audit to shipped components.



Design system architecture — building the token layer, component library, and Figma-to-code pipeline from scratch.



Auth0 UI component design — error states, access control screens, and authentication flow components.



Developer handoff — delivering production-ready React + Tailwind component packages with full documentation.

Tools & technologies

Figma

Tailwind Tokens

Tailwind CSS v3

React

NexJS

Poppins - Typography

Auth0 SDK

Okta SDK

SCSS

VSCode

Claude Code

Kiro

Gemini

Bitbucket

Jira

02

Design system foundation

Why the system came first

The biggest mistake in enterprise product redesigns is jumping to screen-level work before the foundation is in place. You fix one screen, move to the next, and by the time you come back the first screen already drifted. I designed the token and component system before touching a single production screen — so that every screen built on top of it would inherit consistency automatically.

Token architecture

I built a Tokens Studio-compatible token system structured in four layers, all wired through to a Tailwind config extension and CSS custom properties:

- 1 Color tokens** — BeyondID brand red (#D04B2F) as the primary accent. Semantic status colors — icc-blue for informational, icc-green for success/passing, icc-amber for warnings, icc-red for critical states. A neutral scale (neutral-0 to neutral-900) for surfaces, borders, and text hierarchy.
- 2 Typography tokens** — DM Sans for all UI text (300/400/500/600 weights). DM Mono for data labels, IDs, code, and progress counters. Minimum 11px for any label, 13px for body, 15px for lead text, 18px+ for headings.
- 3 Spacing & layout tokens** — 4px base grid. Sidebar width (280px), topbar height (56px), and card padding as named tokens — ensuring consistent spatial rhythm across all screens without guessing.
- 4 Component tokens** — Border radius values (6px atoms, 10px cards, 14px containers), box shadow levels (card, card-elevated), and border widths (0.5px resting, 1.5px interactive states).

A design token is a design decision recorded once and applied everywhere. The goal wasn't to create a beautiful Figma file — it was to create a system where the right color, spacing, and type style appears automatically, not because a designer remembered to apply it.

Component migration strategy

After laying the token foundation, I migrated components in atomic order: primitive atoms first (buttons, inputs, badges, chips), then molecules (cards, form groups, status rows), then full page layouts. The Health Check Assessment became the reference validation screen — the first page to run the full system end-to-end and prove it worked before expanding to other screens.

03 Health Check Assessment redesign

The Health Check Assessment is ICC's compliance module — it guides Okta tenant administrators through a structured set of security and configuration questions mapped to NIST IAL standards. Completing the assessment generates a formal compliance report for the client

organization. This screen carries regulatory weight: an incomplete or confusing submission can result in failed audits.

The challenges I set out to solve

No answer confirmation

Clicking a radio option produced no visual feedback. In a compliance audit, users had no way to confirm their answers were registered — leading to re-clicks and data uncertainty.

No progress visibility

A 16-question mandatory assessment with no persistent progress indicator. Users couldn't see how far they'd come or how much remained — a significant completion anxiety risk.

Navigation breakdown

The sidebar was too narrow to display full category names. Critical navigation labels were cut off, breaking wayfinding in a structured, multi-category workflow.

Ambiguous actions

Footer action buttons were icon-only — no labels, no tooltips. Users couldn't distinguish between "Attach evidence," "Add comment," and "Clear answer."

Missing context

In multi-tenant environments, submitting an assessment against the wrong tenant is a real operational risk. The current tenant was not persistently visible anywhere on screen.

Unclear filter system

The Manual vs Automated filter had no visible effect on the question list and was not labeled. The distinction between check types — critical for IAM engineers — was invisible.

My design process

1

Domain mapping before wireframes

I spent time mapping the IAM domain: Okta multi-tenant architecture, NIST IAL compliance tiers (IAL1/IAL2/IAL3), and the real-world difference between manual checks (performed by an admin) and automated checks (polled by the system). This understanding directly shaped every design decision — I knew what information needed to always be visible, and what could be on-demand.

2

Structural layout redesign

Rebuilt the layout with a wider, readable sidebar (280px — enough to display full category names). Introduced a persistent topbar showing global progress ("4 of 16 answered"). Designed a question card architecture that cleanly separates question content, type badge, and contextual action controls into distinct visual zones.

3

Closing the feedback loop

Designed a blue-highlighted row selection state with an inline "✓ Saved" confirmation chip that appears immediately when an answer is selected. This single change directly resolved the most critical trust failure in the original screen — users now have unambiguous, instant confirmation that every compliance answer is recorded.

4

Two-level progress system

Added both a global progress bar in the topbar (macro: "how close am I to finishing?") and per-category mini progress bars in the sidebar (micro: "have I completed Access Management?"). Compliance workflows require both levels of awareness — administrators need to track progress at the full-assessment level and the section level simultaneously.

5

Tenant context anchoring

Added a persistent "Okta — Tenant 1 · Active" chip pinned to the top of the sidebar. In a multi-tenant environment, this context needs to be always visible, not a detail the user has to hunt for before submitting.

ORIGINAL SCREEN CHALLENGES

- No visual confirmation on answer selection
- No overall progress indicator
- Sidebar labels cut off mid-word
- Icon-only footer buttons
- Tenant context not visible
- Manual/Automated filter invisible

REDESIGNED SCREEN

- Blue highlight + "✓ Saved" chip on every answer
- Global topbar: "4 of 16 answered" always visible
- Full-width sidebar, all category names readable
- Labeled footer actions with icons + text
- Tenant + status chip pinned to sidebar top
- Manual/Automated badge on every question card

Additional enhancements: per-category progress bars in sidebar, collapsible evidence attachment and comment panels, Previous/Next keyboard-friendly footer navigation, and the Manual/Automated filter now has a labeled segment control with visible effect.

04

Manage Report Redesign

The Manage Report page is what clients see after an assessment — compliance outputs, security activity, and tenant health data presented to stakeholders. It's a high-visibility page that directly affects client confidence in BeyondID as a product and a partner. When I reviewed it, the page presented significant visual inconsistencies that needed to be resolved.

The challenges

Inconsistent section headers

Four different section header styles across the same page — different backgrounds, different text colors, no consistent visual rule. The page looked assembled, not designed.

Broken status badge system

Status badges used arbitrary colors with no semantic system. RESOLVED and CRITICAL looked visually similar. Severity levels weren't communicating at a glance.

Accessibility gaps

Multiple text labels throughout the page were below the minimum readable size — failing accessibility standards for a product used in professional compliance contexts.

Inconsistent spacing

Section gaps varied widely with no discernible pattern, making the page feel structurally disorganized rather than informationally dense.

What I redesigned

- 1 Unified section header system** — All section headers standardized to dark navy (#1E293B) with white text across the entire page — one visual rule, regardless of which service or data source the section represents.
- 2 Semantic status badge system** — Built a consistent badge language: RESOLVED/COMPLETED → green (#DCFCE7 / #166534). CRITICAL → red (#DC2626). HIGH → orange. LOW → amber. Each severity level now has a distinct, immediately readable color that communicates without requiring text.
- 3 Typography accessibility pass** — All sub-11px labels raised to 11px minimum. Badge font sizes bumped from 10px to 12px. Page title increased from 16px to 18px. The page now meets WCAG minimum size requirements throughout.
- 4 Normalized spacing** — All section gaps standardized to 24px. Section-internal header-to-content spacing made consistent. A subtle header bottom separator added to improve section definition.
- 5 Tab bar corrected to brand color** — Active tab state changed from off-brand orange to ICC brand blue (#3F59E4) — consistent with every other interactive state across the product.

The Manage Report page is what clients see. Every inconsistency there is a credibility signal — it tells the client the product was assembled, not designed. Bringing it into a coherent visual system was as much about trust as it was about aesthetics.

Because I write production code, my handoffs work differently. I don't deliver Figma files and wait for engineering to interpret them. I deliver working components, already built to spec, that engineering wires to real data and ships. This changes the conversation from "what does this state mean?" to "here's the prop — here's the data shape."

The Health Check Assessment package

For the Health Check Assessment redesign, I produced a complete eight-file React + Tailwind package. Every design decision was encoded directly in the component logic:

TOKEN design-tokens.json — Single source of truth for all ICC color, type, and spacing values.

TOKEN tokens.css — CSS custom properties extracted from the token file. Usable in any project regardless of whether they use Tailwind, Bootstrap, or plain CSS.

TOKEN tailwind.config.js — Full Tailwind v3 theme extension. All token colors, spacing, radius, and shadow values wired to Tailwind utility class names.

REACT QuestionCard.jsx — Self-contained assessment question card. Radio options, Manual/Automated badge, blue selection highlight, "✓ Saved" confirmation chip, expandable evidence and comment panels.

REACT AssessmentSidebar.jsx — Category navigation with collapsible sections, per-category progress bars, tenant context chip, and user identity bar at the bottom.

REACT AssessmentTopbar.jsx — Global progress bar ("4 of 16"), breadcrumb navigation, save draft and submit controls.

REACT HealthCheckAssessment.jsx — Full page assembly with a buildCategories() data helper and wired sample data. Engineering replaces the sample data with the real API response and ships.

docs README.md — Setup guide with font and Tailwind install steps, complete component prop API tables, and usage examples for each component.

The key architectural decision: data layer separation

The most important structural decision in this handoff was separating the data layer from the component internals. The full page assembly accepts a single questions data array. Engineers modify only that array to wire in real assessment questions, categories, and answer options — the components themselves are untouched.

This means the design is resilient to real-world content variation: different question counts per category, variable text lengths, new question types introduced in future sprints. The visual system handles it automatically because it was designed with those variations in mind from the start.

06 Impact & key learnings

What this work delivered for the product

A design foundation that scales

The Tokens Studio token architecture now covers 100% of ICC's color, typography, spacing, and component values. Every future screen is built on a consistent, documented foundation.

Compliance workflow that earns trust

The Health Check Assessment went from an interface where users were unsure if their answers were being saved, to one where every action has clear, immediate confirmation. Critical for a regulatory tool.

Client-facing report professionalized

The Manage Report page now presents a coherent, credible visual system to stakeholders — unified headers, semantic status colors, and accessible typography throughout.

Zero dev handoff friction

Production-ready React components eliminated the translation layer between design and engineering. Ship time decreased because there was nothing to interpret — just data to wire.

Design investment compounded

The token system and component patterns built for the Health Check Assessment were directly reused on the Manage Report page and Auth0 components. One investment, multiple payoffs.

Product feels intentional

Across all screens, ICC now has a shared visual language — consistent color meaning, spatial rhythm, and interaction patterns. It reads as a designed product, not a collection of pages.

Key learnings from this project

Foundation first, always. The most impactful decision I made on this project was building the token and component system before redesigning any screen. It meant every subsequent screen benefitted automatically — consistency wasn't something I had to enforce manually, it was baked into the system.

Building what you design changes everything. Delivering working components instead of static specs removed the biggest risk in any design-to-development process: interpretation. When I hand off a component, the design intent is already encoded in the code — there's nothing to misread.